

GUÍA PARA PENSAR EJERCICIO 1

PASO 1 — Entender qué hace el ejercicio realmente

Antes de escribir nada, tenemos que entender:

- Se recibe **un mensaje desordenado**.
- Hay que convertirlo en un mensaje **limpio y uniforme**.
- Luego hay que **dividirlo en palabras**.
- Después hay que **compararlo con palabras clave**.
- Finalmente, el main tiene que ofrecer un **menú** que permita hacer todo eso de forma ordenada.

PASO 2 — Planificar el método de normalización

La normalización pide:

- quitar espacios de los extremos
- pasar a minúsculas
- eliminar caracteres que no sean letras ni espacio
- evitar espacios repetidos

Conceptos que deberían activarse en la cabeza:

- recorrido carácter a carácter
- uso de `.charAt(i)`
- condiciones que filtran caracteres

- creación de un nuevo string
- último carácter leído (para detectar espacios repetidos)

Preguntas clave para guiar:

1. ¿Cómo sabes si un carácter es una letra entre 'a' y 'z'?
2. ¿Cómo evitar añadir dos espacios seguidos?
3. ¿Cómo construir un string sin usar `replaceAll`?
4. ¿Qué pasa si el mensaje tiene muchos símbolos?
5. ¿Hay que convertir a minúsculas antes o después de filtrar?

Mini-desafío

Describir en una frase el algoritmo que usarías (sin código).

PASO 3 — Planificar el método que extrae palabras

Este método recibe **el mensaje ya normalizado**.

Objetivo: devolver un array EXACTO de palabras.

Preguntas para guiar:

1. Si ya está normalizado, ¿hay símbolos inesperados?
2. ¿Qué método de `String` permite dividir por espacios?
3. ¿Por qué el array final no debe tener huecos ni strings vacíos?
4. ¿Cómo decidir el tamaño final del array?
5. ¿Se puede resolver en dos pasadas?
(una para contar, otra para copiar)

Idea para pensar:

Pensar que es como "filtrar" lo que devuelve `split()`.

PASO 4 — Planificar el método que cuenta palabras clave

Este método recibe:

- array de palabras del mensaje
- array de palabras clave

Concepto central: comparación palabra a palabra.

Preguntas para orientar:

1. ¿Hay que distinguir mayúsculas? (No, porque ya está normalizado).
2. ¿Cuántos bucles se necesitan?
3. ¿Hay que devolver la primera coincidencia, o contarlas todas?
4. ¿Qué incrementa el contador?
5. ¿Qué pasa si una palabra aparece varias veces?

PASO 5 — Diseñar el menú del main

Preguntas guía:

1. ¿Qué información necesita recordar el main entre una opción y otra?
→ mensaje original, mensaje normalizado, array de palabras...
2. ¿Qué pasa si el usuario intenta ver las palabras antes de normalizar?
→ Debe haber un valor inicial o un mensaje de aviso.
3. ¿Cómo organizarías las opciones para evitar repetir código?
4. ¿Qué opciones debe tener el menú?

PASO 6 — Conectar los métodos entre sí

MENSAJE ORIGINAL

↓ normalizar

MENSAJE NORMALIZADO

↓ extraer palabras

ARRAY DE PALABRAS

↓ comparar con claves

RESULTADO FINAL

GUÍA PRÁCTICA EJERCICIO 1

1. MÉTODO: normalizar(String mensaje)

Objetivo:

- trim
- minúsculas
- quitar caracteres no alfabéticos
- quitar espacios repetidos



Pista 1: Recorrido carácter a carácter

Un patrón típico es:

```
for (int i = 0; i < mensaje.length(); i++) {  
    char c = mensaje.charAt(i);  
    // aquí decides si c te interesa o no  
}
```



Pista 2: Filtrar letras

Recordad que un carácter es letra si:

```
c >= 'a' && c <= 'z'
```



Pista 3: Construir un nuevo String

No hay que modificar el original, sino crear uno nuevo:

```
String resultado = "";
```

y luego ir concatenando:

```
resultado += c;
```

Pista 4: Espacios repetidos

```
boolean ultimoFueEspacio = false;

if (c == ' ') {
    if ( /* condición necesaria */ ) {
        // añadir espacio
    }
    // actualizar bandera
} else {
    // añadir letra normal
    // actualizar bandera
}
```

Objetivo: descubrir la condición necesaria.

2. MÉTODO: extraerPalabras(String mensajeNormalizado)

Objetivo:

- dividir por espacios
- descartar elementos vacíos
- crear un array exacto

Pista 1: usar split

```
String[] trozos = mensajeNormalizado.split(" ");
```

Pista 2: contar palabras válidas

Plantilla sin rellenar:

```
int contador = 0;

for (int i = 0; i < trozos.length; i++) {

    if (!trozos[i].equals( /* ??? */ )) {

        contador++;

    }

}
```

¿Qué comparación evita los vacíos?

Pista 3: crear array final exacto

Este es el patrón, ¿Cuál es su contenido?

```
String[] palabras = new String[contador];

int pos = 0;

for (int i = 0; i < trozos.length; i++) {

    if ( /* condición adecuada */ ) {

        palabras[pos] = trozos[i];

        pos++;

    }

}
```

3. MÉTODO:

contarPalabrasClave(String[] palabras, String[] claves)

Objetivo:

- doble bucle
- comparar con equals
- acumular un contador

Pista 1: doble for

Estructura básica sin rellenar:

```
int contador = 0;

for (int i = 0; i < palabras.length; i++) {
    for (int j = 0; j < claves.length; j++) {
        if ( /* comparación adecuada */ ) {
            // acción sobre contador
        }
    }
}
```

¿Cuál es la comparación correcta?

RESUMEN

- Usa un bucle para normalizar.
- Usa `split` para partir palabras.
- Usa dos bucles anidados para contar claves.
- Usa un menú con `do-while` y `switch` para organizar el flujo.
- Cada método resuelve **una fase del proceso**.

